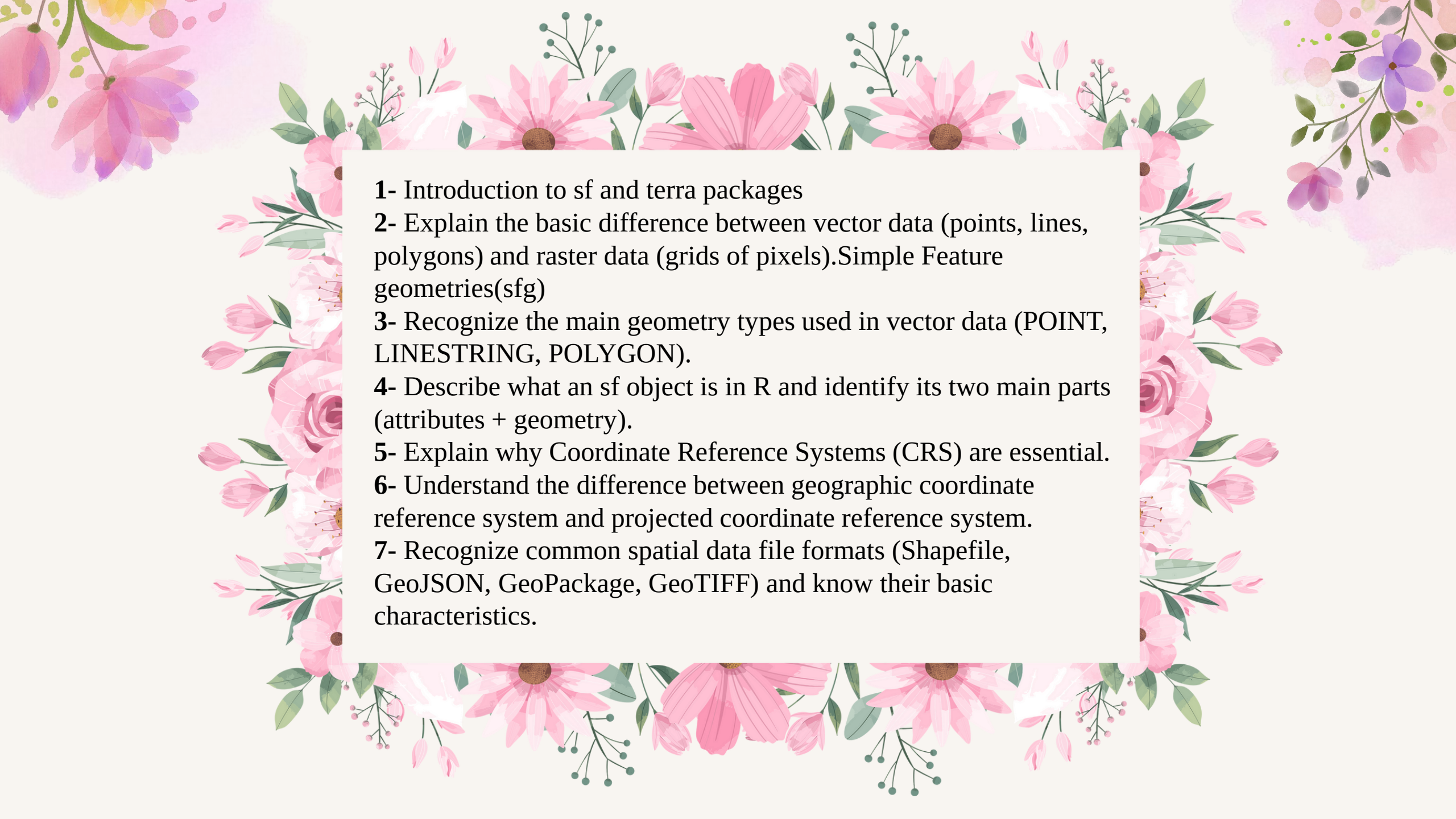
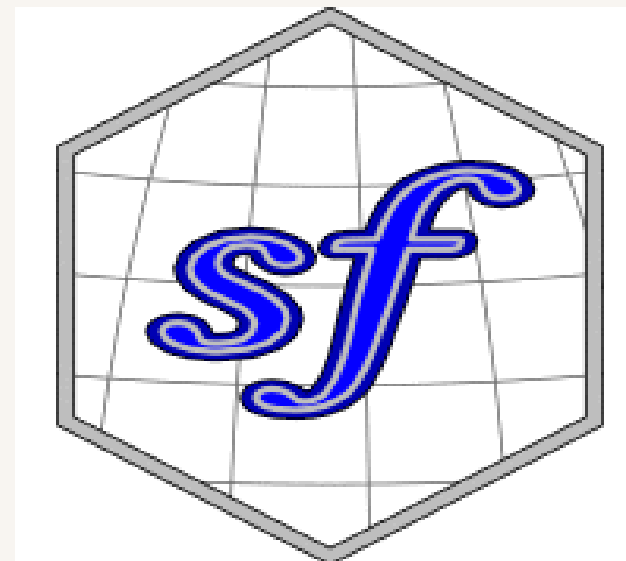




Geocomputation with R
Season3:
2 Geographic data in R

by: Dina Yazdani

- 
- 1- Introduction to sf and terra packages
 - 2- Explain the basic difference between vector data (points, lines, polygons) and raster data (grids of pixels). Simple Feature geometries(sfg)
 - 3- Recognize the main geometry types used in vector data (POINT, LINESTRING, POLYGON).
 - 4- Describe what an sf object is in R and identify its two main parts (attributes + geometry).
 - 5- Explain why Coordinate Reference Systems (CRS) are essential.
 - 6- Understand the difference between geographic coordinate reference system and projected coordinate reference system.
 - 7- Recognize common spatial data file formats (Shapefile, GeoJSON, GeoPackage, GeoTIFF) and know their basic characteristics.





Vector Data and Raster Data



Two Flavors of Spatial Data: Vector vs. Raster

When computers store map information, they usually use one of two main strategies: Vector or Raster.

1. Vector Data – Drawing with Precision

- **Points:** Single coordinate pairs (X, Y or Lon/Lat) representing specific locations.

Examples: Cities, individual trees, customer addresses. (*sf geometry: POINT / MULTIPOINT*)

- **Linestrings:** Sequences of connected coordinate points forming a line.

Examples: Rivers, roads, railways, hiking trails. (*sf geometry: LINESTRING / MULTILINESTRING*)

- **Polygons:** Closed sequences of connected points defining an area.

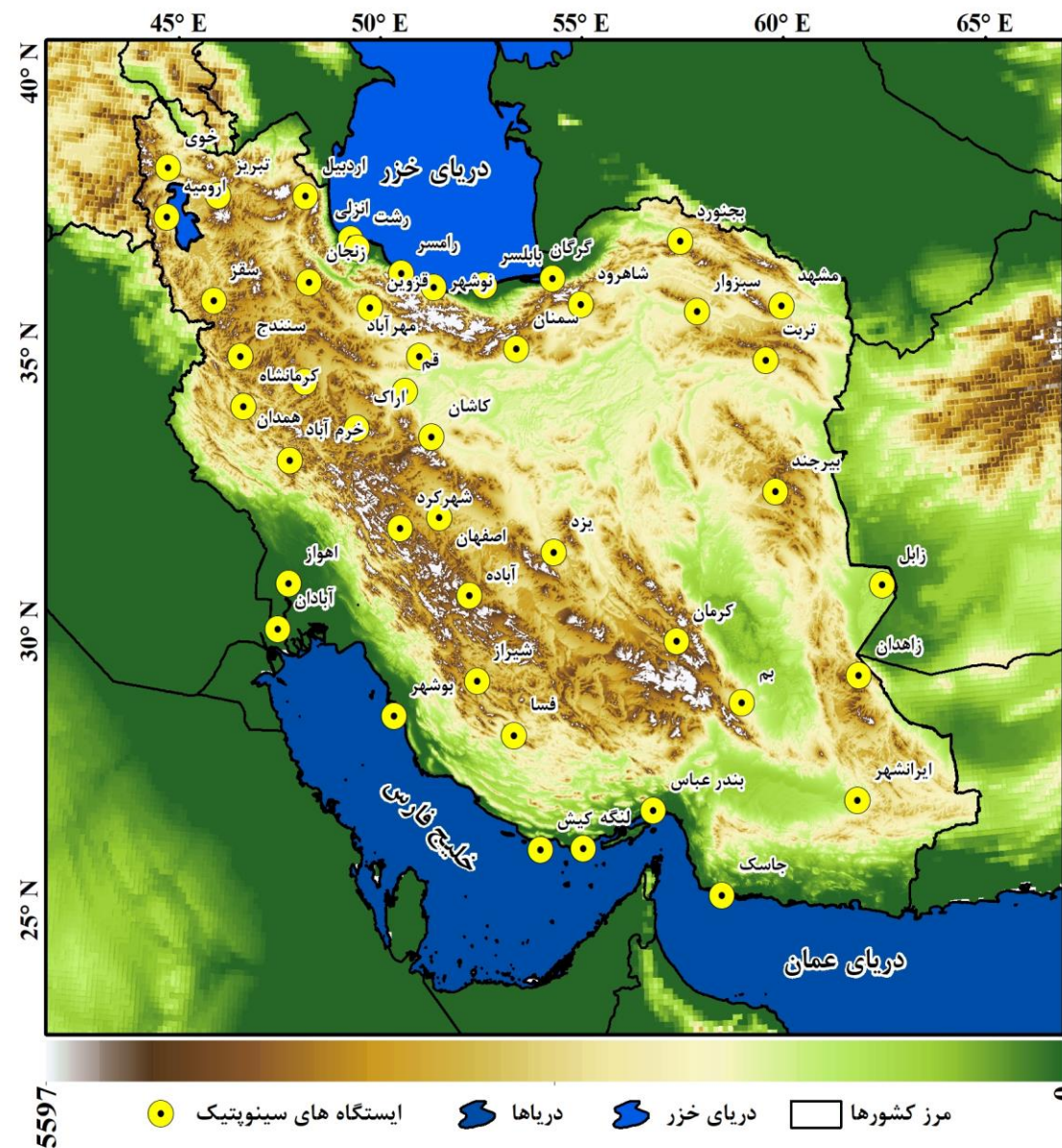
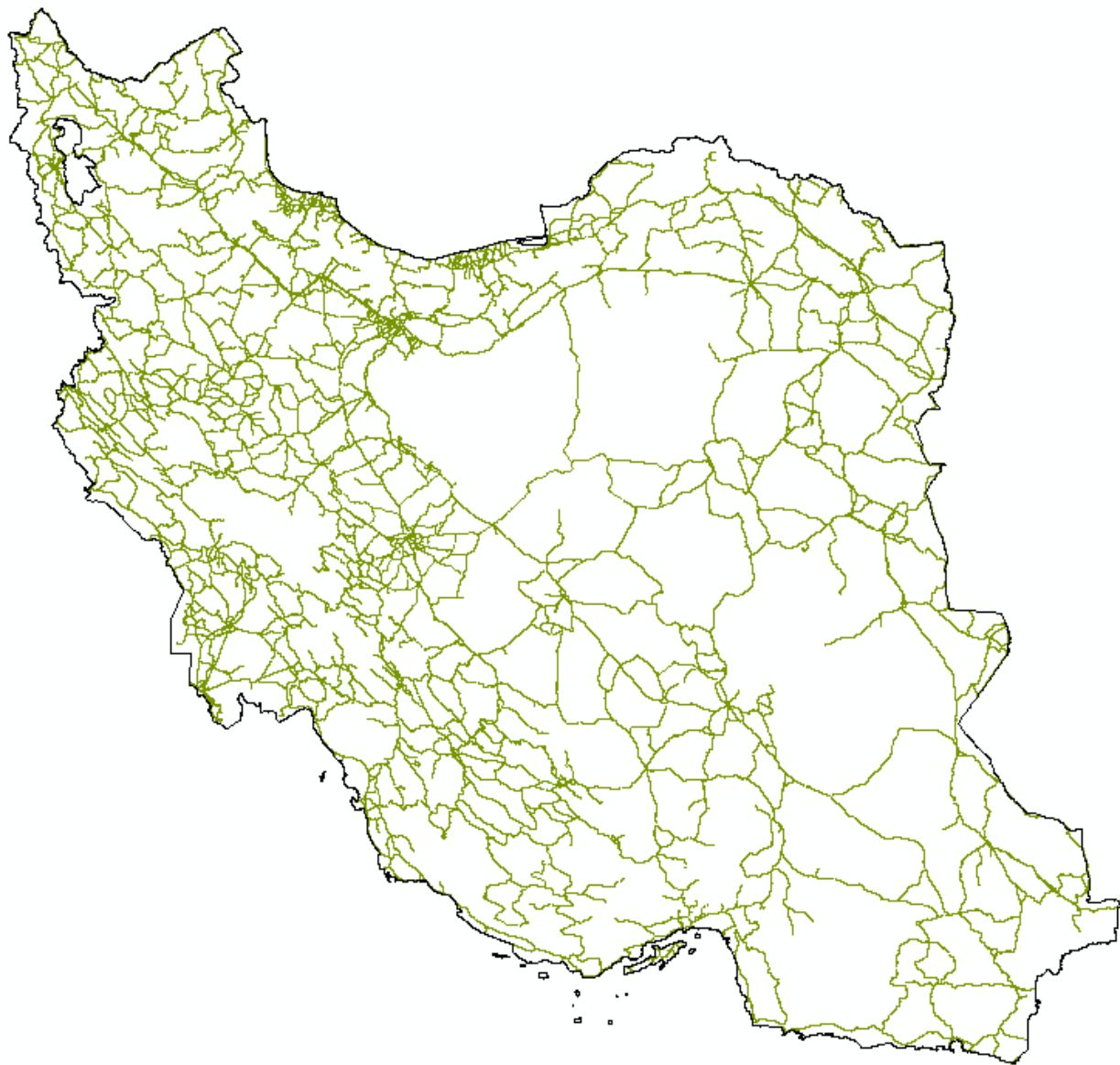
Examples: Country outlines, state boundaries, lake shapes, building footprints. (*sf geometry: POLYGON / MULTIPOLYGON*)

2. Raster Data – The World as a Grid

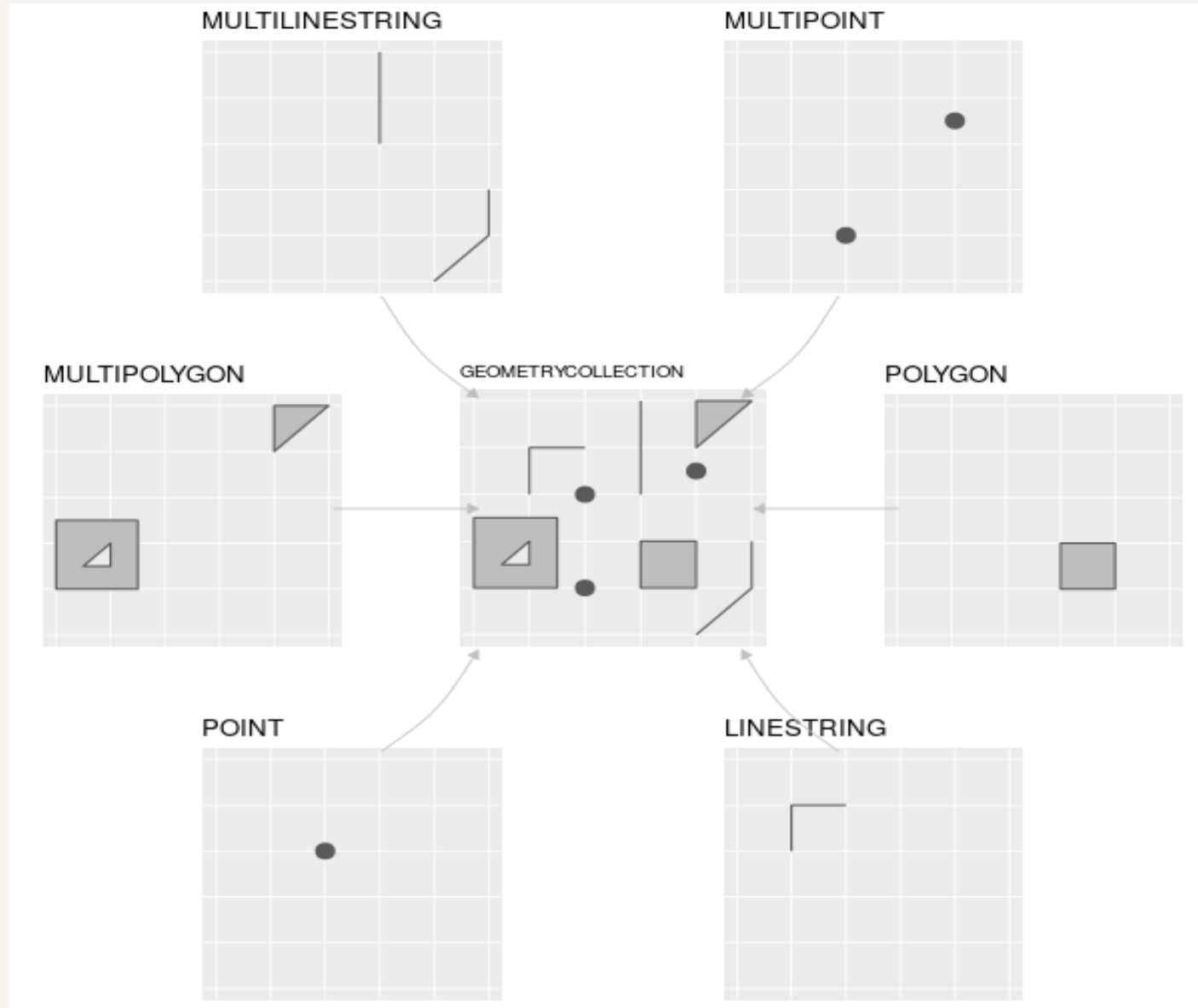
- Think of raster data like a digital photograph made of colored squares (pixels).

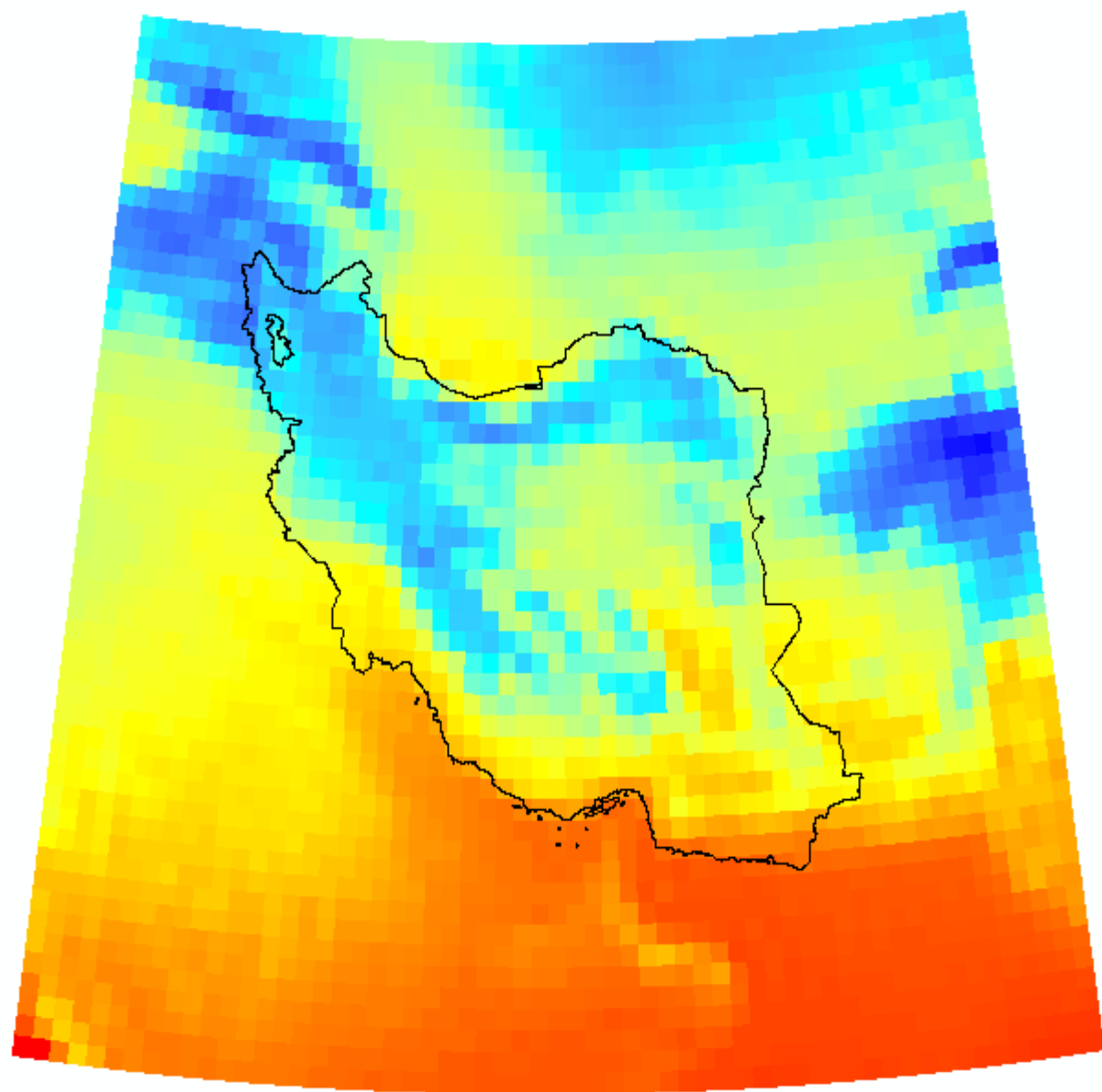
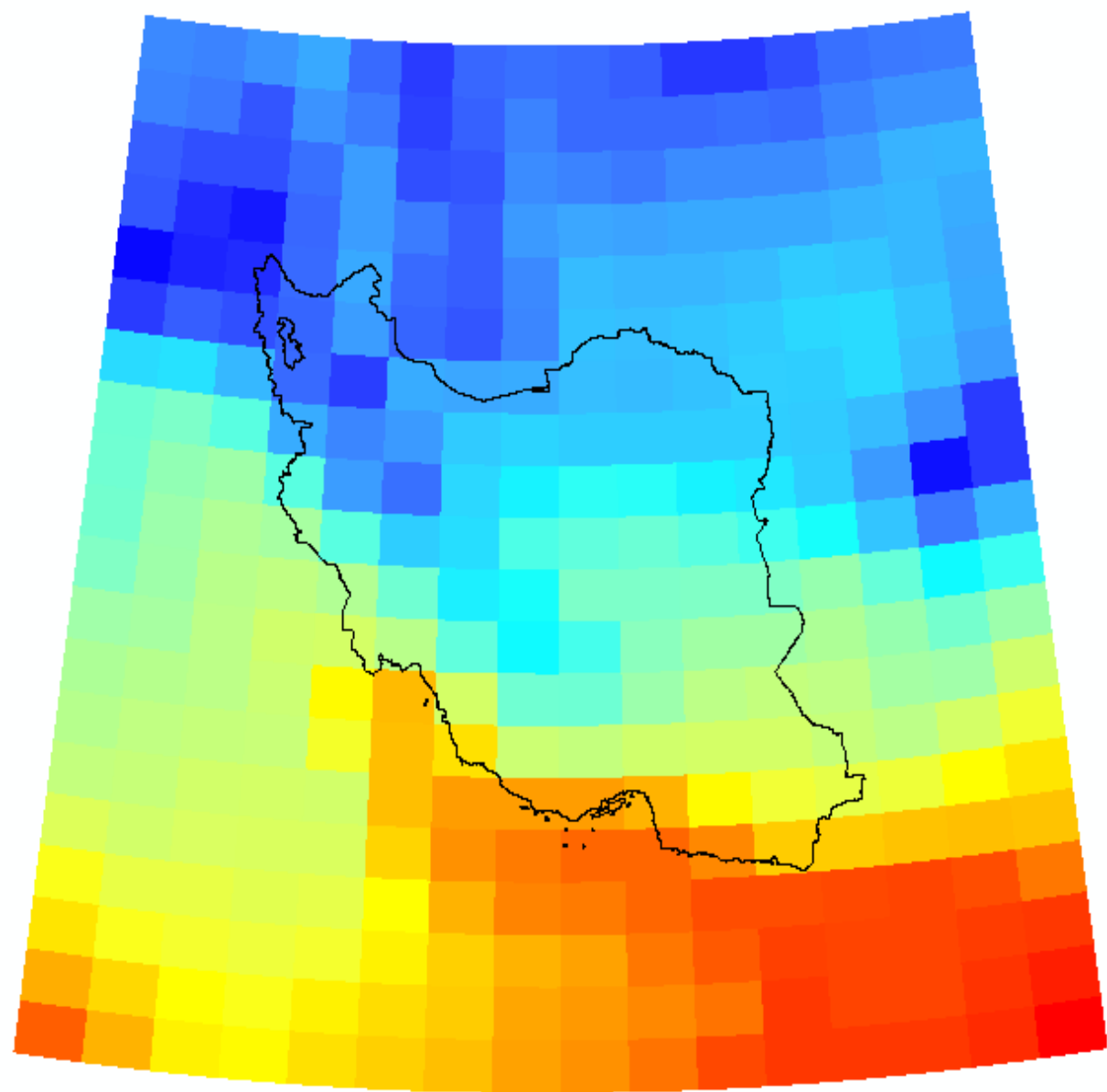
Each pixel covers a specific geographic area and stores a value (e.g., elevation m, temperature °C, or a categorical land-cover code).

- Standard tool in R: **terra**



Recognize the main geometry types





Coordinate Reference Systems (CRS) Revisited

The Core Problem: Earth = Round(ish), Map/Screen = Flat. How do we represent locations accurately?

1. **Datum** How do we mathematically model the Earth's imperfect shape?

- The Earth isn't a perfect sphere; it bulges at the equator and is slightly squashed (an ellipsoid or spheroid). Different datums use slightly different ellipsoids that best fit the Earth's shape either globally or for a specific region.
- A datum also defines an origin point (like the Earth's center) and orientation.
- Common Example: WGS84 (World Geodetic System 1984). This is a global datum used by GPS systems. It's very common for storing data.
- Analogy: Choosing which slightly imperfect orange model you'll use as your reference before you start peeling.

2. **Projection** – How do we mathematically translate coordinates from the curved datum surface onto a flat map?

- This is the “flattening the orange peel” part. All projections introduce some kind of distortion – you can't perfectly flatten a curved surface without stretching, tearing, or squashing something.

Different projections are designed to preserve different properties accurately, while distorting others:

- **Conformal Projections** (e.g., Mercator, Lambert Conformal Conic): Preserve local shapes and angles correctly. Great for navigation charts. But they distort areas, especially far from the center line (Mercator makes Greenland look HUGE!).
- **Equal-Area Projections** (e.g., Albers Equal Area, Mollweide, Eckert IV): Preserve area correctly across the map. Excellent for comparing the size of regions (e.g., for thematic maps showing density). But they distort shapes, angles, and distances.
- **Equidistant Projections:** Preserve distance correctly from one or two central points to all other points. But distort shape and area elsewhere.
- **Azimuthal Projections:** Preserve direction (azimuth) correctly from a central point. Often used for polar maps or airline routes.
- *Analogy:* The specific way you choose to cut and flatten your orange peel – some ways keep the pieces looking the right shape but stretch the size, others keep the size right but make the shapes look weird.

Identifying CRSs: EPSG Codes

Remembering the complex details of datums and projections is hard! Luckily, standard CRSs are often identified by unique codes. The most common system is the EPSG code, managed by the International Association of Oil & Gas Producers (originally the European Petroleum Survey Group).

- **EPSG:4326:** The code for the very common WGS84 Latitude/Longitude system. You'll see this everywhere! It uses degrees as units. Great for storing global data, often not ideal for analysis or mapping specific regions due to distortion and degree units.
- **EPSG:3857:** The code for the Web Mercator projection often used by online map tiles (like Google Maps, OpenStreetMap). Preserves shape locally but heavily distorts area near the poles. Uses meters as units (but these meters are only truly accurate near the equator!).
- **UTM Zones (e.g., EPSG:32632 for UTM Zone 32N):** Universal Transverse Mercator divides the world into 60 narrow zones. Each zone has its own projected CRS designed to be accurate (low distortion in shape, area, distance) within that specific zone. Uses meters. Excellent for local or regional mapping and analysis.
- **National/Continental Grids (e.g., EPSG:5070 for USA Albers Equal Area, EPSG:28992 for Dutch National Grid):** Many countries or continents have official projected CRSs designed for accuracy within their boundaries. Often use meters or feet.



Finding CRS Information (epsg.io):

How do you find the right EPSG code or details about one?

- Online Resource: <https://epsg.io/> is an invaluable website. You can search by:
 - EPSG code (e.g., search “4326”)
 - Geographic location (e.g., search “Netherlands” or “California”)
 - Projection name (e.g., search “Albers Equal Area”)
- It provides details about the projection, datum, units, accuracy, and importantly, the EPSG code you can use in R (e.g., in `st_transform(crs = “EPSG:XXXX”)`).

Key Takeaway Recap: Every spatial dataset has a CRS (Datum + Projection). You must ensure data layers share the same CRS for alignment and analysis. Use `st_crs()` to check and `st_transform()` to change CRS, often using standard EPSG codes found via resources like epsg.io. Choose projections suitable for your map’s purpose and geographic area!

Common Spatial File Formats

Format	Type	Typical Extension(s)	Notes
Shapefile	Vector	.shp + others	Legacy multi-file format; keep .shp, .shx, .dbf (attributes) and .prj (CRS) together.
GeoJSON	Vector	.geojson, .json	Modern text-based single-file; web-friendly.
GeoPackage	Vector/Raster	.gpkg	Single SQLite-based file; can store multiple layers; highly recommended.
GeoTIFF	Raster	.tif, .tiff	Standard for rasters; embeds CRS and other metadata.
CSV	Table (optionally spatial)	.csv	Needs conversion to <i>sf</i> with <code>st_as_sf()</code> and explicit CRS.